РЕШЕН ИЗПИТ 6 — С ПЪЛНИ ОБЯСНЕНИЯ

Базови Програмни Езици — C (plain C)

Изпит 6: Магазин — Продукти | 100 точки

Теми: Задача 1 пише в БИНАРЕН файл (не текстов!) | filterAbovePrice връща НОВ масив | bin→txt двуфазно конвертиране

ВНИМАНИЕ: Задача 1 ползва fopen("ab") — бинарно добавяне, не текстово!

Задача 3 е най-сложна — два файла, два цикъла, двуфазен алгоритъм.

СЕКЦИЯ 0 — ПРОЧИТ И АНАЛИЗ

КОД / ИЗРАЗ	КАКВО ОЗНАЧАВА ЗА НАС
`Име на продукт (1 до 30 символа)`	char name[31]; — 30 + 1 = 31
`ID (код) на продукт (цяло число)`	int id; — обикновен int
`Срок на годност ГГГГ.ММ.ДД`	char expiry[11]; — 10 символа + \0 = 11
`Цена на продукт (реално число)`	float price; — %.2f при printf
`Създайте бинарен файл с име 'in.bin'`	fopen("ab") за добавяне — Задача 1 е БИНАРНА!
`Функцията да добавя в масива И в бинарния файл`	Два различни fwrite извиквания? Не — fopen в addProduct
`Функцията връща масив с цена > зададената`	Нов динамичен масив с realloc в тялото на функцията
`Принтира последователно от двата файла`	Binary Item: ... Text Item: ... редуващо се

СЕКЦИЯ 1 — ПЪЛЕН КОД (gcc -Wall -std=c99)

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char  name[31];    /* до 30 символа + '\0' */
    int   id;          /* код на продукта */
    char  expiry[11];  /* ГГГГ.ММ.ДД = 10 + '\0' */
    float price;       /* цена */
} Product;

/* ── ЗАДАЧА 1: addProduct() — пише В БИНАРЕН файл! ── */
Product* addProduct(Product *arr, int *count) {
    (*count)++;
    arr = (Product*)realloc(arr, (*count) * sizeof(Product));
    if (!arr) { fprintf(stderr, "realloc failed\n"); exit(1); }
    Product *p = &arr[(*count) - 1];
    printf("Продукт: ");  scanf("%30s",  p->name);
    printf("ID:      ");  scanf("%d",   &p->id);
    printf("Годност: ");  scanf("%10s",  p->expiry);
    printf("Цена:    ");  scanf("%f",   &p->price);

    /* Записваме в БИНАРЕН файл — 'ab' = append binary */
    FILE *f = fopen("in.bin", "ab");  /* НЕ "a" — бинарно! */
    if (f) {
        size_t w = fwrite(p, sizeof(Product), 1, f);
        if (w != 1) printf("Предупреждение: запис неуспешен!\n");
        fclose(f);
    }
    return arr;
}

/* ── ЗАДАЧА 2: filterAbovePrice() — връща НОВ масив ── */
Product* filterAbovePrice(Product *arr, int count,
                          float minPrice, int *newCount) {
    Product *result = NULL;
    *newCount = 0;
    for (int i = 0; i < count; i++) {
        if (arr[i].price > minPrice) {  /* СТРОГО по-голямо */
            (*newCount)++;
            result = (Product*)realloc(result, (*newCount) * sizeof(Product));
            if (!result) { fprintf(stderr, "realloc failed\n"); exit(1); }
            result[(*newCount) - 1] = arr[i];  /* копира целия struct */
        }
    }
    return result;  /* може да е NULL ако нищо не е намерено! */
}

/* ── ЗАДАЧА 3: convertBinToText() — двуфазен алгоритъм ── */
void convertBinToText(const char *binFn, const char *txtFn) {
    FILE *fb = fopen(binFn, "rb");
    if (!fb) { printf("Грешка: %s\n", binFn); return; }
    FILE *ft = fopen(txtFn, "w");
    if (!ft) { fclose(fb); printf("Грешка: %s\n", txtFn); return; }

    /* — ФАЗА 1: binary → text ──────────────────────────── */
    Product p;
    while (fread(&p, sizeof(Product), 1, fb) == 1) {
```

```c
        printf("Binary Item:\n");
        printf("Name: %s\n", p.name);
        printf("ID: %d\n", p.id);
        printf("Expire Date: %s\n", p.expiry);
        printf("Price: %.2f\n", p.price);
        fprintf(ft, "%s;%d;%s;%.2f\n",
                p.name, p.id, p.expiry, p.price);
    }
    fclose(fb); fclose(ft);

    /* — ФАЗА 2: четем от text файла ———————————————— */
    ft = fopen(txtFn, "r");
    if (!ft) return;
    char name[31]; int id; char expiry[11]; float price;
    while (fscanf(ft, "%30[^;];%d;%10[^;];%f\n",
           name, &id, expiry, &price) == 4) {
        printf("Text Item:\n");
        printf("Name: %s\n", name);
        printf("ID: %d\n", id);
        printf("Expire Date: %s\n", expiry);
        printf("Price: %.2f\n", price);
        printf("---------------\n");
    }
    fclose(ft);
}

int main() {
    Product *arr = NULL; int count = 0;
    arr = addProduct(arr, &count);
    arr = addProduct(arr, &count);

    int filteredCount = 0;
    Product *filtered = filterAbovePrice(arr, count, 15.0f, &filteredCount);
    if (filtered) {
        for (int i = 0; i < filteredCount; i++)
            printf("%s - %.2f\n", filtered[i].name, filtered[i].price);
        free(filtered);
    }

    convertBinToText("in.bin", "outTxt");
    free(arr); return 0;
}
```

СЕКЦИЯ 2 — ЗАДАЧА 1: БИНАРЕН ФАЙЛ ПРИ ДОБАВЯНЕ

Ключовата разлика от Изпит 5: там текстовият файл се отваря с fopen("a"). Тук Задача 1 записва в БИНАРЕН файл in.bin с fopen("ab") — append binary.

```
[NE]  fopen("in.bin", "a")    // "a" е за текстов append — НЕ за бинарен!
[OK]  fopen("in.bin", "ab")   // "ab" = append binary — ПРАВИЛНО за Изпит 6
```

КОД	ОБЯСНЕНИЕ
`fopen("in.bin", "ab")`	"ab" = append binary. При всяко addProduct добавяме нов запис в края без изтриване.
`fwrite(p, sizeof(Product), 1, f);`	Записваме САМО новия елемент p. &arr[count-1] е същото като p тук.
`if (w != 1)` `printf("Предупреждение");`	fwrite връща броя записани елементи. Проверката е задължителна!

Задача 2: filterAbovePrice() — нов динамичен масив

Функцията СЪЗДАВА нов масив и го връща. Важно: caller-ът (main) отговаря за free(). Ако нищо не отговаря на условието, функцията връща NULL.

КОД	ОБЯСНЕНИЕ
`Product *result = NULL;`	Новият масив. NULL = празен. realloc(NULL, n) == malloc(n).
`*newCount = 0;`	Изходен параметър — записваме в него колко намерихме.
`result = realloc(result, ...)`	Разширяваме result с 1 за всеки намерен елемент.
`result[...] = arr[i];`	Копираме ЦЕЛИЯ struct с =. Всички полета се копират.
`return result;`	Може да е NULL ако нищо не е намерено. Caller проверява: if (filtered) { ... }

Задача 3: двуфазното конвертиране

Алгоритъмът е в две фази защото трябва да принтираме Binary Item и Text Item редуващо се — но двата файла се обхождат независимо. Ако ги правехме в един цикъл, нямаше да можем да разделим Binary от Text.

```
/* Структура на изхода: */
Binary Item:
Name: Meat  ID: 123  Expire: 2022.09.22  Price: 23.23
Text Item:
Name: Meat  ID: 123  Expire: 2022.09.22  Price: 23.23
```

```
----------------
/* ФАЗА 1: чети от bin, пиши в txt, принтирай Binary Item */
/* ФАЗА 2: чети от txt, принтирай Text Item */
```

КОД	ОБЯСНЕНИЕ
`FILE *fb = fopen(binFn, "rb");`	Отваряме binary за четене.
`FILE *ft = fopen(txtFn, "w");`	"w" = пишем нов текстов файл. Не "wb" — текстовият файл не е бинарен.
`if (!ft) { fclose(fb); return; }`	Ако txt не се отвори — затваряме fb ПРЕДИ return! Не само return;
`fclose(fb); fclose(ft);`	Затваряме ДВАТА файла след фаза 1. Данните трябва да са на диска за фаза 2.
`ft = fopen(txtFn, "r");`	Повторно отваряне за четене. ft е вече NULL след fclose.
`fscanf(ft, "%30[^;];%d;%10[^;];%f\n", ...) == 4`	Четем 4 полета разделени с ';'. == 4 = успешно прочетен ред.

Грешки характерни за Изпит 6

```
[NE]  fopen("in.bin", "a")      // текстов append — данните се разрушават!
[OK]  fopen("in.bin", "ab")     // бинарен append — правилно

[NE]  return result;  // без free(filtered) в main() — memory leak!
[OK]  if (filtered) { /* ползваме */ free(filtered); }  // caller освобождава

[NE]  if (!ft) { return; }       // fb не се затваря преди return!
[OK]  if (!ft) { fclose(fb); return; }  // затвори fb преди return
```